Premier University Chittagong



Comparative Analysis of Regression Models for Stock Market Prediction

by

Pankaj Rudra

ID: 2104010202242

Mohammed Hobaibul Islam

ID: 2104010202262

Dipannita Roy prima

ID: 1703310201467

September 2024

Student Name : Pankaj Rudra
ID : 2104010202242
: Mohammed Hobaibul Islam
: 2104010202262
: Dipannita Roy prima
: 1703310201467

Session : Spring 2024

Supervisor Name : Adiba Ibnat Hossain
Designation : Professor

Course Title : Artificial Intelligence

Tentative Title **: Comparative Analysis of Regression Models for Stock Market Prediction**

Table of Contents

List of Figures **ii**

1 Introduction 1
2 Motivation 1
3 Objectives 1
4 Related Works 1
5 Methodology 2
5.1 Data Preparation 2
5.2 Exploratory Data Analysis 2
5.3 Model Training 3
5.4 Model Evaluation 3
6 Implementation 4
6.1 Data Preprocessing 4
6.2 Feature Selection 4
6.3 Data Splitting 4
6.4 Model Training and Evaluation 4
7 Visualization 5
8 Performance Evaluation 8
9 Code Script 8
10 Future Works 13
11 Conclusion 13
12 Team Contributions 14
12.1 Pankaj Rudra (ID: 2104010202242) 14
12.2 Mohammed Hobaibul Islam (ID: 2104010202262) 14
12.3 Dipannita Roy Prima (ID: 1703310201467) 14

List of Figures

7.1 Correlation Heatmap of the data 5
7.2 Pyplot View of the data (1) 5
7.3 Pyplot View of the data (2) 5
7.4 Actual Values vs. Predicted Values of Linear Regression model . . 6
7.5 Actual Values vs. Predicted Values of Decision Tree model 6
7.6 Actual Values vs. Predicted Values of Random Forest Regression model 6
7.7 Actual Values vs. Predicted Values of Gradient Boosting Regression model 7
7.8 Actual Values vs. Predicted Values of SVR (Support Vector Regressor) model 7
7.9 Combined plot of actual vs. predicted values for all models 7

1 Introduction

This lab report aims to explore and evaluate various regression models to predict stock prices. The stock price data of Netflix (NFLX) is used to train and test the models. The report details the methodology, implementation, and performance evaluation of five regression models: Linear Regression, Decision Tree Regressor, Random Forest Regressor, Gradient Boosting Regressor, and Support Vector Regressor..

2 Motivation

Accurate stock price prediction is crucial for investors to make informed decisions. Leveraging machine learning models can enhance prediction accuracy and provide a competitive edge in financial markets. This study aims to identify which regression model performs best in predicting the opening price of Netflix stock based on historical data.

3 Objectives

- To preprocess the stock price data and make it suitable for machine learning models.
- To implement and train multiple regression models.
- To evaluate and compare the performance of these models.
- To visualize the correlation between different features and the actual vs. predicted values

4 Related Works

Stock price prediction has been a significant area of research in financial engineering. Various models, including statistical methods and machine learning algorithms, have been applied to predict stock prices. Previous studies have shown

that ensemble methods like Random Forest and Gradient Boosting often outperform individual models due to their ability to handle overfitting and provide better generalization.

5 Methodology

The methodology for predicting stock prices involves several key steps, including data preparation, exploratory data analysis, model training, and evaluation. Below is a detailed description of each step:

5.1 Data Preparation

The dataset used in this study is sourced from Netflix (NFLX) stock prices and is read from a CSV file. The following steps are performed in data preparation:

- Data Loading: The data is loaded into a Pandas DataFrame using the `pd.read_csv()` function.
- Missing Values Handling: The dataset is inspected for missing values, and appropriate measures are taken to handle them.
- Data Conversion: Non-numeric columns are converted to numeric types where necessary. For instance, the 'Volume' column is converted from string to float after removing commas.

5.2 Exploratory Data Analysis

Exploratory Data Analysis (EDA) is performed to understand the relationships between different features. The following methods are used:

- Correlation Heatmap: A correlation heatmap is plotted to visualize the relationships between numeric features. This helps in identifying which features are strongly correlated with each other.
- Pairplot: A pairplot is generated to provide a quick overview of the pairwise relationships between features. This visual tool helps in understanding the distribution and relationships of data points.

5.3 Model Training

Several regression models are implemented and trained on the dataset:

- Feature Selection: The independent variables (features) include 'High', 'Low', 'Close', and 'Volume'. The dependent variable (target) is 'Open'.
- Train-Test Split: The dataset is split into training and testing sets using the `train_test_split()` function from scikit-learn. A test size of 10% is used to evaluate model performance.
- Model Initialization and Training:
  - **Linear Regression:** A Linear Regression model is initialized and trained on the training data.
  - **Decision Tree Regressor:** A Decision Tree Regressor is initialized and trained.
  - **Random Forest Regressor:** A Random Forest Regressor is initialized and trained.
  - **Gradient Boosting Regressor:** A Gradient Boosting Regressor is initialized and trained.
  - **Support Vector Regressor:** A Support Vector Regressor is initialized and trained.

5.4 Model Evaluation

The performance of each model is evaluated using the R^2 score, which measures how well the model explains the variance in the target variable. Additionally, predictions from each model are compared to actual values by plotting them together. This visual comparison helps in assessing the accuracy and reliability of each model.

6 Implementation

6.1 Data Preprocessing

1. Loading Data: The dataset is loaded using Pandas.
2. Handling Missing Values: The dataset is checked for missing values.
3. Data Cleaning: The 'Volume' column is converted to float by removing commas.
4. Correlation Heatmap: A heatmap is plotted to visualize the correlation between numeric features.
5. Pairplot: A pairplot is created to overview relationships between features.

6.2 Feature Selection

1. Features: 'High', 'Low', 'Close', 'Volume'
2. Target: 'Open'

6.3 Data Splitting

- The dataset is split into training (90%) and testing (10%) sets.

6.4 Model Training and Evaluation

1. Linear Regression: Trained and evaluated on the test set.
2. Decision Tree Regressor: Trained and evaluated on the test set.
3. Random Forest Regressor: Trained and evaluated on the test set.
4. Gradient Boosting Regressor: Trained and evaluated on the test set.
5. Support Vector Regressor: Trained and evaluated on the test set.

The performance of each model is evaluated using the R-squared score.

7 Visualization

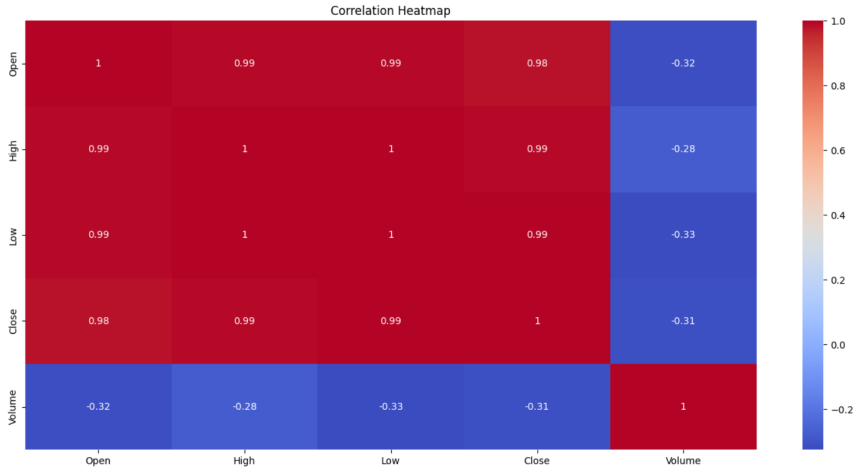


Figure 7.1: Correlation Heatmap of the data

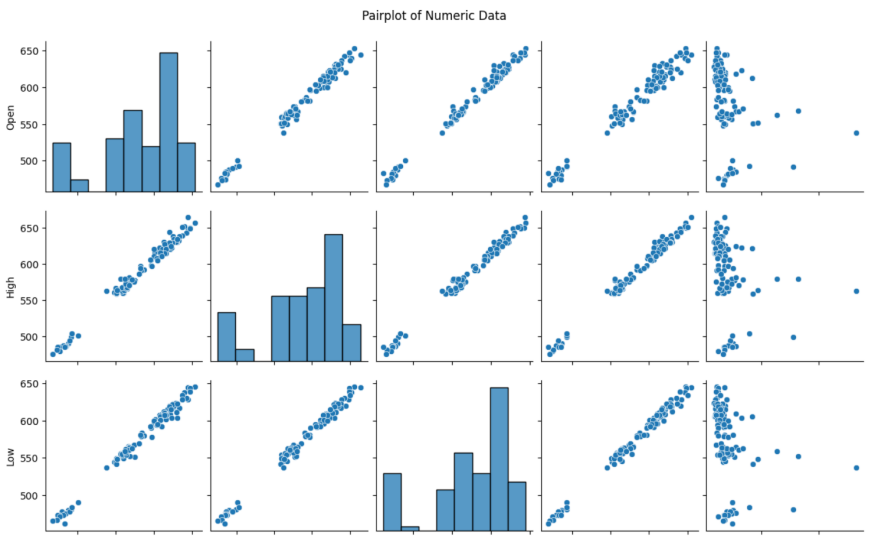


Figure 7.2: Pyplot View of the data (1)

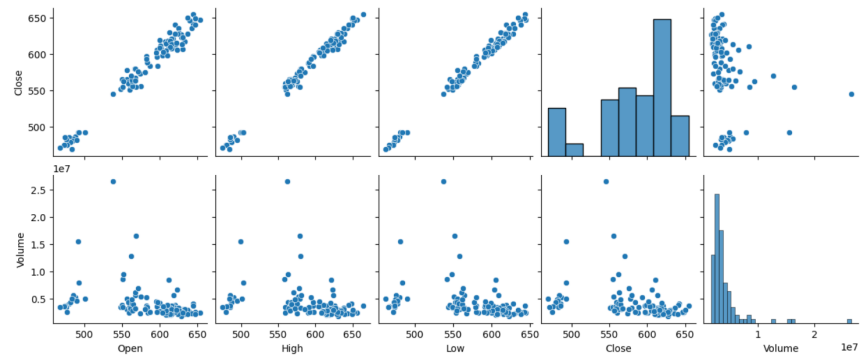


Figure 7.3: Pyplot View of the data (2)

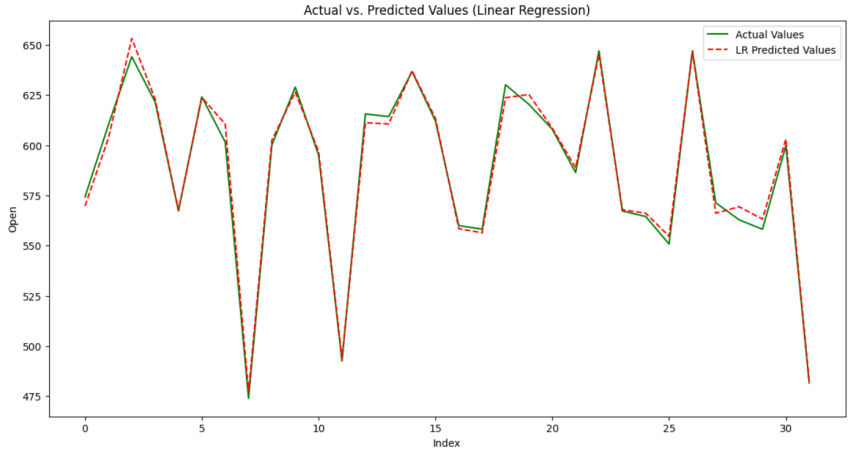


Figure 7.4: Actual Values vs. Predicted Values of Linear Regression model

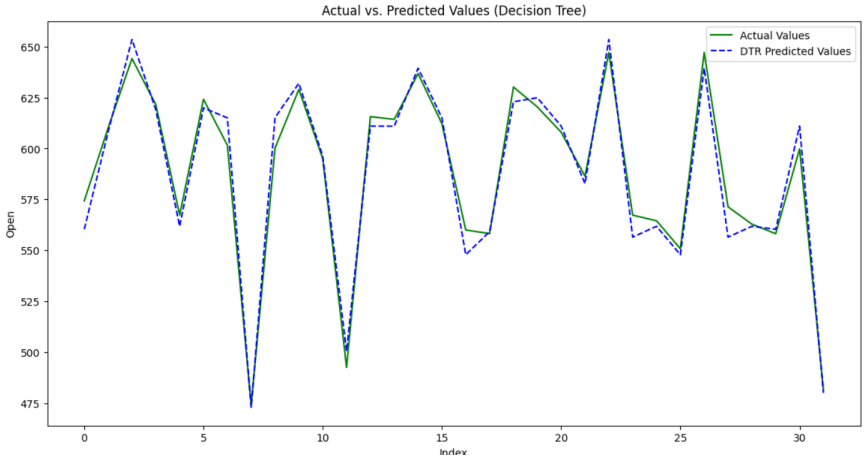


Figure 7.5: Actual Values vs. Predicted Values of Decision Tree model

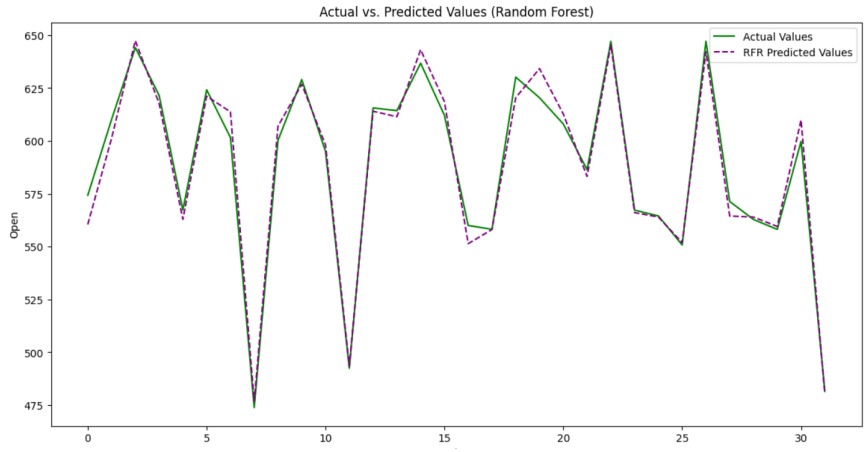


Figure 7.6: Actual Values vs. Predicted Values of Random Forest Regression model

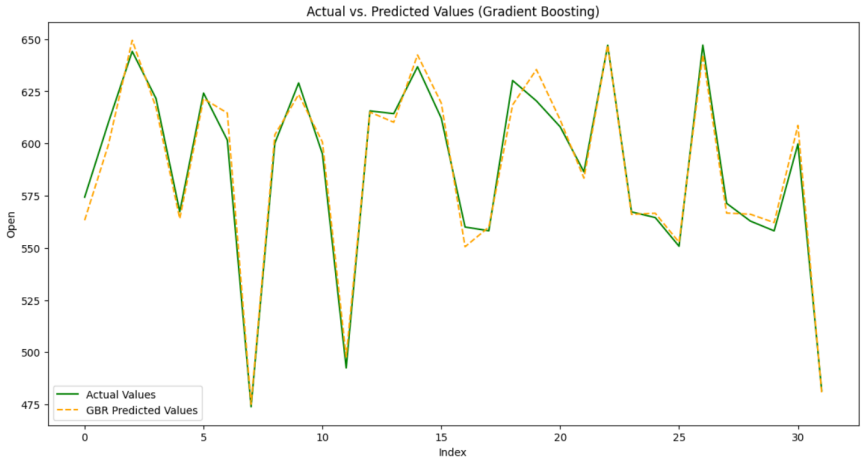


Figure 7.7: Actual Values vs. Predicted Values of Gradient Boosting Regression model

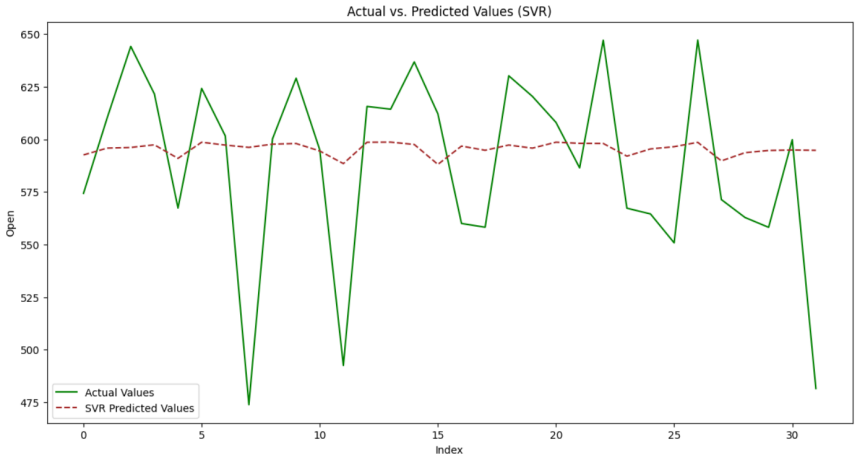


Figure 7.8: Actual Values vs. Predicted Values of SVR (Support Vector Regressor) model

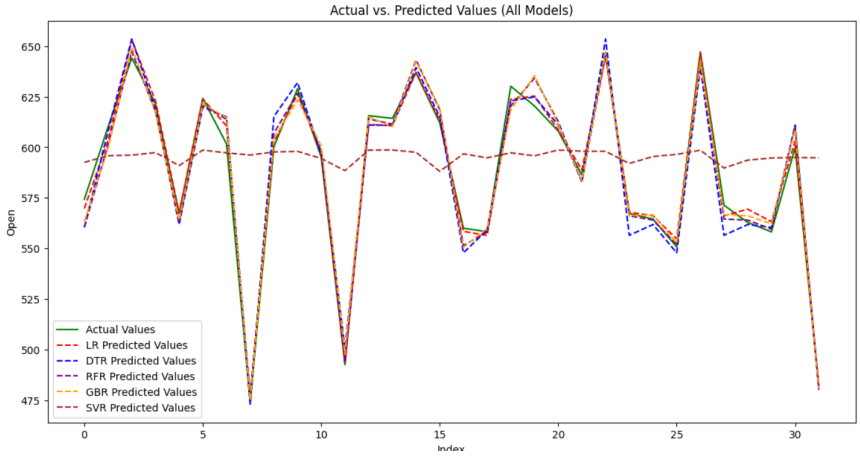


Figure 7.9: Combined plot of actual vs. predicted values for all models

8 Performance Evaluation

The R-squared scores of each model on the test set are as follows:

- Linear Regress: (R^2 =0.997)
- Decision Tree Regressor: ($R^2 = 0.988$)
- Random Forest Regressor: ($R^2 = 0.997$)
- Gradient Boosting Regressor: ($R^2 = 0.996$)
- Support Vector Regressor: ($R^2 = 0.985$)

9 Code Script

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.linear_model import LinearRegression
from sklearn.tree import DecisionTreeRegressor
from
sklearn.ensemble
GradientBoostingRegressor
from sklearn.svm import SVR
import
RandomForestRegressor,
from sklearn.model_selection import train_test_split
# Reading the CSV file from the local file system
data = pd.read_csv('Download Data - STOCK_US_XNAS_NFLX.csv')
# Displaying the first few rows of the dataset
print(data.head())
# Checking for any missing values in the dataset
print(data.isnull().sum())
# Converting the 'Volume' column to float by removing commas
data['Volume'] = data['Volume'].str.replace(',', '').astype(float)
```

```
# Selecting only numeric columns for correlation heatmap
numeric_data = data[['Open', 'High', 'Low', 'Close', 'Volume']]
# Plotting the correlation heatmap
plt.figure(figsize=(17, 8))
cor = sns.heatmap(numeric_data.corr(), annot=True, cmap='coolwarm')
plt.title('Correlation Heatmap')
plt.show()
# Plotting pairplot for a quick overview of relationships
sns.pairplot(numeric_data)
plt.suptitle('Pairplot of Numeric Data', y=1.02)
plt.show()
# Selecting features (independent variables) and target (dependent
variable)
x = data[['High', 'Low', 'Close', 'Volume']]
y = data['Open']
# Displaying the first few rows of the features and target
print(x.head())
print('=======================================')
print(y.head())
# Splitting the dataset into training and testing sets
x_train, x_test, y_train, y_test = train_test_split(x, y,
test_size=0.1, random_state=0)
# Initializing and fit the Linear Regression model
LR = LinearRegression()
LR.fit(x_train, y_train)
print("Linear Regression Model Score on Test Data:", LR.score(x_test,
y_test))
# Plotting the predicted values vs. actual values for Linear
Regression
plt.figure(figsize=(14, 7))
y_pred_LR = LR.predict(x_test)
plt.plot(y_test.values, label='Actual Values', color='green')
```

```
plt.plot(y_pred_LR, label='LR Predicted Values', color='red',
linestyle='dashed')
plt.xlabel('Index')
plt.ylabel('Open')
plt.title('Actual vs. Predicted Values (Linear Regression)')
plt.legend()
plt.show()
# Initializing and fit the Decision Tree Regressor model
DTR = DecisionTreeRegressor(random_state=0)
DTR.fit(x_train, y_train)
print("Decision Tree Model Score on Test Data:", DTR.score(x_test,
y_test))
# Plotting the predicted values vs. actual values for Decision Tree
plt.figure(figsize=(14, 7))
y_pred_DTR = DTR.predict(x_test)
plt.plot(y_test.values, label='Actual Values', color='green')
plt.plot(y_pred_DTR, label='DTR Predicted Values', color='blue',
linestyle='dashed')
plt.xlabel('Index')
plt.ylabel('Open')
plt.title('Actual vs. Predicted Values (Decision Tree)')
plt.legend()
plt.show()
# Initializing and fit the Random Forest Regressor model
RFR = RandomForestRegressor(random_state=0)
RFR.fit(x_train, y_train)
print("Random Forest Model Score on Test Data:", RFR.score(x_test,
y_test))
# Plotting the predicted values vs. actual values for Random Forest
plt.figure(figsize=(14, 7))
y_pred_RFR = RFR.predict(x_test)
plt.plot(y_test.values, label='Actual Values', color='green')
```

```
plt.plot(y_pred_RFR, label='RFR Predicted Values', color='purple',
linestyle='dashed')
plt.xlabel('Index')
plt.ylabel('Open')
plt.title('Actual vs. Predicted Values (Random Forest)')
plt.legend()
plt.show()
# Initializing and fit the Gradient Boosting Regressor model
GBR = GradientBoostingRegressor(random_state=0)
GBR.fit(x_train, y_train)
print("Gradient
Boosting
GBR.score(x_test, y_test))
Model
Score
on
Test
Data:",
# Plotting the predicted values vs. actual values for Gradient
Boosting
plt.figure(figsize=(14, 7))
y_pred_GBR = GBR.predict(x_test)
plt.plot(y_test.values, label='Actual Values', color='green')
plt.plot(y_pred_GBR, label='GBR Predicted Values', color='orange',
linestyle='dashed')
plt.xlabel('Index')
plt.ylabel('Open')
plt.title('Actual vs. Predicted Values (Gradient Boosting)')
plt.legend()
plt.show()
# Initializing and fit the Support Vector Regressor model
SVR_model = SVR()
```

```
SVR_model.fit(x_train, y_train)
print("SVR Model Score on Test Data:", SVR_model.score(x_test,
y_test))
# Plotting the predicted values vs. actual values for SVR
plt.figure(figsize=(14, 7))
y_pred_SVR = SVR_model.predict(x_test)
plt.plot(y_test.values, label='Actual Values', color='green')
plt.plot(y_pred_SVR, label='SVR Predicted Values', color='brown',
linestyle='dashed')
plt.xlabel('Index')
plt.ylabel('Open')
plt.title('Actual vs. Predicted Values (SVR)')
plt.legend()
plt.show()
# Testing data for prediction
Test_data = [[47.37, 44.59, 46.61, 30826543.0]]
prediction_LR = LR.predict(Test_data)
prediction_DTR = DTR.predict(Test_data)
prediction_RFR = RFR.predict(Test_data)
prediction_GBR = GBR.predict(Test_data)
prediction_SVR = SVR_model.predict(Test_data)
# Printing the predictions
print("Linear Regression Prediction for Test Data:", prediction_LR)
print("Decision Tree Prediction for Test Data:", prediction_DTR)
print("Random Forest Prediction for Test Data:", prediction_RFR)
print("Gradient Boosting Prediction for Test Data:", prediction_GBR)
print("SVR Prediction for Test Data:", prediction_SVR)
# Plotting the predicted values vs. actual values for each model
together
plt.figure(figsize=(14, 7))
plt.plot(y_test.values, label='Actual Values', color='green')
plt.plot(y_pred_LR, label='LR Predicted Values', color='red',
```

```
linestyle='dashed')
plt.plot(y_pred_DTR, label='DTR Predicted Values', color='blue',
linestyle='dashed')
plt.plot(y_pred_RFR, label='RFR Predicted Values', color='purple',
linestyle='dashed')
plt.plot(y_pred_GBR, label='GBR Predicted Values', color='orange',
linestyle='dashed')
plt.plot(y_pred_SVR, label='SVR Predicted Values', color='brown',
linestyle='dashed')
plt.xlabel('Index')
plt.ylabel('Open')
plt.title('Actual vs. Predicted Values (All Models)')
plt.legend()
plt.show()
```

10 Future Works

Future work could include:

- Exploring other machine learning algorithms like Neural Networks and
- Incorporating additional features such as technical indicators and macroeconomic variables.
- Performing hyperparameter tuning to improve model performance.

11 Conclusion

This study demonstrates the implementation and evaluation of five regression models for predicting Netflix's stock opening price. The results indicate that Linear Regression and Random Forest Regressor perform exceptionally well with high R-squared scores, followed closely by Gradient Boosting Regressor. These models can effectively predict stock prices and can be further improved with additional features and hyperparameter tuning.

12 Team Contributions

12.1 Pankaj Rudra (ID: 2104010202242)

Pankaj Rudra was responsible for the following tasks:

- Data Preparation: Loading the dataset, cleaning, and preprocessing the data.
- Exploratory Data Analysis: Generating correlation heatmaps and pairplots to understand the relationships in the data.
- Implementation of Regression Models: Setting up and training the Linear Regression and Decision Tree Regressor models.
- Performance Evaluation: Evaluating and visualizing the performance of Linear Regression and Decision Tree Regressor models.

12.2 Mohammed Hobaibul Islam (ID: 2104010202262)

Mohammed Hobaibul Islam contributed to the following aspects:

- Implementation of Regression Models: Setting up and training the Random Forest Regressor and Gradient Boosting Regressor models.
- Performance Evaluation: Evaluating and visualizing the performance of Random Forest Regressor and Gradient Boosting Regressor models.
- Prediction Analysis: Conducting predictions using the Random Forest Regressor and Gradient Boosting Regressor models and analyzing results.

12.3 Dipannita Roy Prima (ID: 1703310201467)

Dipannita Roy Prima focused on:

- Implementation of Regression Models: Setting up and training the Support Vector Regressor model.

- Performance Evaluation: Evaluating and visualizing the performance of the Support Vector Regressor model.
- Consolidation and Reporting: Combining results from all models, preparing the plots, and writing the report sections.
- Document Preparation: Formatting the report and integrating all members' contributions into the final document.